LCRSF: A Liveness-Coupled Resilient Scheduling Framework for Dynamic Edge Robotics

Your Name, *Member, IEEE*, Second Author, *Fellow, IEEE*, and Third Author, *Member, IEEE*

***Abstract*—Collaborative edge robot swarms in unstructured environments face an inherent conflict between complex task orchestration and harsh physical constraints, characterized by communication intermittency, thermal heterogeneity, and irreversible node degradation. Existing scheduling paradigms fail to address this triad effectively: Cloud-native orchestrators rely on heavy strong-consistency protocols unsuited for fragile edge links; while End-to-End Deep Reinforcement Learning (DRL) schedulers suffer from black-box uncertainty, lacking deterministic safety guarantees and struggling with sparse failure rewards in real-world deployments. Traditional robotic middlewares also lack liveness guarantees, frequently leading to zombie tasks and resource deadlocks. To bridge this gap, we propose LCRSF, a decentralized, liveness-coupled resilient scheduling framework. Unlike static DAG schedulers, we introduce a Liveness-Entangled Dependency Model that atomically binds task lifecycles to the stochastic survival probabilities of physical nodes. We formulate the scheduling challenge as a Chance-Constrained Stochastic Optimization problem, incorporating a novel thermal-aware utility function handled by a DRL-Augmented Adaptive Bidding mechanism. This bi-level architecture uses DRL as a meta-controller to dynamically tune safety margins and utility weights based on implicit environmental states, while the underlying Utility-based Temporal Auction Algorithm (UTAA) ensures hard real-time execution. Instead of relying on global knowledge, we introduce a Feedback-driven Adaptive Thresholding mechanism in UTAA, using a PI controller to stabilize communication complexity to asymptotic $\mathcal{O}(1)$. System-level implementation using C++17 and Zenoh demonstrates that LCRSF achieves millisecond-level fault recovery and reduces network overhead by 90% compared to state-of-the-art baselines.**

***Index Terms*—Edge Robotics, Resilient Scheduling, Liveness Coupling, Chance-Constrained Optimization, DRL Meta-Control, Thermal Awareness.**

I. Introduction

WITH the rapid convergence of the Internet of Things (IoT) and Embodied Artificial Intelligence (AI), collaborative edge robot swarms are transitioning from structured industrial environments to unstructured, open-world scenarios. In these domains, robots no longer function as isolated execution units but form mobile ad-hoc networks (MANETs) to share perceptual data and collaboratively execute complex task pipelines. However, unlike distributed systems deployed in stable cloud data centers, edge robotic networks are characterized by high stochasticity and unreliability. Nodes are strictly constrained by battery capacity and thermal limits. We term the irreversible degradation of node availability—caused by energy depletion, thermal throttling, or component wear—as *Physical Entropy Increase*. Furthermore, compact robotic platforms face a critical **Computation-Mobility Trade-off**. The waste heat generated by high-torque actuators during rapid movement propagates to computing units via chassis thermal conduction. This creates complex thermal dynamics where a robot sprinting to improve connectivity may thermally saturate, rendering it unsuitable for intensive computation. Traditional schedulers that treat compute and move as orthogonal resources fail to capture this physical coupling. In traditional distributed computing assumptions, nodes are typically considered persistent or capable of being restarted by a centralized controller. In the edge context, this assumption collapses. A single node's *silent failure* often causes dependent downstream tasks to hang indefinitely—a phenomenon we term "Zombie Tasks"—triggering resource deadlocks and potentially paralyzing the entire system workflow.

A. Motivation: Theoretical Mismatches in Existing Paradigms

Migrating existing solutions to mobile edge robot networks faces severe theoretical mismatches due to strict physical constraints. These challenges are manifested in the following dimensions:

1) Resource Overhead and Network Fragility of Strong Consistency: Cloud-native architectures typically rely on strong consistency protocols to maintain the global cluster state. However, these consensus mechanisms require frequent network interaction and substantial CPU/memory overhead. For resource-constrained embedded robots like Raspberry Pi and Jetson Nano, this imposes a heavy burden, contending for computational resources required by critical business processes. Furthermore, edge ad-hoc networks suffer from frequent network partitioning due to robot mobility. Once the Master node becomes unreachable, centralized clusters fall into an unavailable state, failing to meet the strict *survivability* requirements of robot swarms.

2) The Uncertainty of End-to-End Learning: While Deep Reinforcement Learning (DRL) has shown promise in optimizing long-term rewards, pure end-to-end DRL schedulers operate as "black boxes." They lack **deterministic safety guarantees** and suffer from the *Sim-to-Real gap*, where policies trained in simulation fail under unseen physical disturbances. Moreover, in highly reliable systems where failures are rare (sparse rewards), pure DRL agents often converge to suboptimal lazy policies that never migrate tasks. LCRSF adopts a different approach: we use a rigorous **Model-based** core for safety while allowing DRL to adaptively tune parameters, ensuring the system never violates physical limits.

This work was supported by [Funding Agency]. (Corresponding author: Your Name.)

The source code and datasets involved in this work are open-sourced at: [Anonymized for Blind Review]

3) Lifecycle Mismatch in Static Dependency Models: Existing DAG schedulers are primarily designed for batch processing semantics, where logic is based on "Finish-to-Start" data dependencies. However, robotic applications are inherently *Streaming Services*. Upstream processes, such as camera drivers, must not only "finish" initialization but also maintain *liveness* throughout the entire task session. Traditional schedulers lack awareness of this *Lifecycle Coupling*. If an upstream node experiences a silent failure, downstream nodes often wait indefinitely for a specific completion signal that never arrives, devolving into zombie tasks that deadlock hardware resources.

4) Inadequacy of Timeout-based Passive Recovery: Fault handling in traditional distributed systems is typically passive and agnostic to the physical state of the nodes. Reliance on TCP or application-layer heartbeats for fault detection creates an unacceptably long window of lost control for robotic systems operating on millisecond-level control loops. Moreover, existing retry policies often assume faults are transient software errors, ignoring that edge faults frequently stem from the *exhaustion of physical resources*. Blindly rescheduling tasks to the same class of nodes can trigger repeated failures, leading to system oscillation.

B. Main Contributions

To tackle these challenges, this paper proposes **LCRSF**, a decentralized, liveness-coupled resilient scheduling framework. LCRSF redefines the orchestration paradigm from data consistency to control flow liveness, bridging the gap between theoretical optimization and system-level execution. Our main contributions are summarized as follows:

1) **Liveness-Entangled Dependency Model:** We introduce a novel dependency model tailored for edge robotic networks. In contrast to existing DAG-based schedulers that focus solely on data consistency, our model atomically binds the computational lifecycle of dependent tasks with the physical health of upstream nodes via the proposed Distributed Task Unit (DTU). By enforcing liveness entanglement constraints, we theoretically eliminate the zombie task phenomenon and resource deadlocks caused by silent node failures in dynamic ad-hoc clusters.
2) **Chance-Constrained Stochastic Optimization Formulation:** We show that the resilient scheduling problem can be formalized as a **Chance-Constrained Utility Maximization (CCUM)** problem. By integrating *Survival Analysis* theory, we implicitly model node failures as stochastic processes with time-varying hazard rates. This allows us to explicitly quantify the physical entropy risk and transform the scheduling challenge into a stochastic optimization problem subject to strict liveness chance constraints, ensuring system reliability under uncertainty.
3) **Feedback-Driven $\mathcal{O}(1)$ Temporal Auction:** To solve the NP-Hard CCUM problem in real-time, we propose the **Utility-based Temporal Auction Algorithm (UTAA)**. We design a bi-level architecture where a lightweight DRL **Meta-Controller** adapts utility weights to implicit environmental states. Crucially, we overcome the "Global Knowledge Paradox" by incorporating a **PID-based Feedback Control** loop. This allows nodes to adaptively tune their pruning thresholds based on the number of bids in previous rounds, ensuring communication complexity remains at asymptotic $\mathcal{O}(1)$ without knowing the global node count N.
4) **Control-Data Decoupled Asynchronous Reactive Runtime:** To support the millisecond-level response required by the theoretical model, we implement a high-performance system runtime. We construct a **Control-Data Decoupled Architecture** that physically isolates high-frequency signaling from high-bandwidth data streams. Furthermore, we overcome the performance bottlenecks of traditional middleware through **User-Space Zero-Copy** communication and **Kernel-Level Watchdog Binding**. Extensive experiments on a physical testbed and large-scale simulations demonstrate that LCRSF reduces the Mean Time To Recovery (MTTR) to 1.2s and maintains a 96% task completion rate under network jitter, significantly outperforming ROS 2 and Reinforcement Learning baselines.

II. Related Work

Dynamic task orchestration in edge computing has attracted significant attention. Existing solutions can be categorized into three main streams: learning-based approaches, heuristic optimization, and security-aware frameworks.

A. Learning-based & Game-Theoretic Approaches

To address environmental dynamics, recent works have leveraged Deep Reinforcement Learning and Game Theory. Chen *et al.* modeled the service migration problem as a Markov Decision Process (MDP) and proposed a Twin-Delayed DDPG-based algorithm to minimize long-term cloud load. Similarly, Zhang *et al.* designed a TC-TD3 algorithm for urban rail transit to allocate heterogeneous resources dynamically. In game theory, Asheralieva *et al.* combined Bayesian games with Lagrange Coded Computing (LCC) to balance privacy and resilience in multi-access edge computing. **Critique:** While effective in cloud-edge scenarios, pure DRL agents typically suffer from *heavy inference overhead* and *slow convergence*, requiring thousands of epochs to adapt to topology changes. Furthermore, their "black-box" nature lacks deterministic latency guarantees required for safety-critical robotic faults. Redundancy-based methods like LCC incur high computation and communication costs, which are often prohibitive for battery-powered micro-robots. **Unlike these approaches, LCRSF uses DRL only for parameter adaptation (Meta-Control), relying on a deterministic optimization core for safety.**

B. Heuristic & Meta-heuristic Optimization

To reduce computational complexity, heuristic algorithms are widely adopted. Liu *et al.* proposed an Adaptive

Task Offloading Algorithm (ATOA) that categorizes tasks based on rule-based policies (e.g., transmission-dominated vs. computation-dominated). Kumar *et al.* utilized the Whale Optimization Algorithm (WOA) combined with fuzzy logic controllers to optimize energy consumption and makespan in Industrial IoT environments. **Critique:** Rule-based heuristics lack theoretical guarantees on optimality and struggle to adapt to stochastic node failures. Meta-heuristics like WOA rely on stochastic search, introducing unpredictable iterative delays. Crucially, these works often assume sufficient control plane bandwidth, overlooking the risk of *broadcast storms* in large-scale ad-hoc clusters.

C. Uniqueness of LCRSF

Table I provides a qualitative comparison of feature sets across representative frameworks. LCRSF is the only framework that simultaneously addresses physical liveness coupling and communication scalability while ensuring hard real-time recovery.

III. System Model & Architecture

To address the inherent conflict between consistency and availability in Mobile Ad-hoc Networks (MANETs), LCRSF constructs a decentralized probabilistic orchestration paradigm. Unlike traditional layered architectures that abstract away hardware details, we adopt a **Physics-First** design philosophy. This section is structured bottom-up: We first model the underlying *Physical Entropy* governing node behavior (Sec. III-B). On top of this, we build a resilient *Logical Architecture* featuring role fluidity and decoupled communication (Sec. III-C). Finally, we introduce the *Distributed Task Unit (DTU)* as the semantic carrier that binds these layers together (Sec. III-D).

A. Notations

Key notations used throughout this paper are summarized in Table II.

B. Physics-Informed Node Abstraction

In unstructured environments, the assumption of persistent, identical computing nodes collapses. We model the edge cluster as a set of heterogeneous autonomous agents $\mathcal{N} = \{N_1, N_2, \ldots, N_k\}$, explicitly formalizing their degradation dynamics.

1) Thermal-Mobility Dynamics Model: To rigorously capture the *Physical Entropy*, we formulate the thermal evolution of an edge node. Unlike servers in air-conditioned data centers, edge robots face a critical **Computation-Mobility Trade-off**. The temperature evolution $T_k(t)$ is governed by a modified Newton's Law of Cooling that accounts for electromotive waste heat:

$$\begin{aligned}\frac{dT_k}{dt} = &\underbrace{\alpha_{cpu} \cdot P_{load}(t)}_{\text{Compute Heat}} + \underbrace{\alpha_{\mathbf{motor}} \cdot |\mathbf{v_k(t)}|}_{\text{Mobility Heat}} \\ &- \underbrace{k_{cool}^k \cdot (T_k(t) - T_{amb})}_{\text{Dissipation}}\end{aligned} \tag{1}$$

Where:

- $P_{load}(t)$ is the computational power consumption.
- $|\mathbf{v_k(t)}|$ is the robot's velocity. The term $\alpha_{motor} \cdot |v_k(t)|$ captures the heat generated by actuators transferring to the computing chassis. This creates a physical coupling: a robot sprinting to improve connectivity may thermally saturate its CPU, rendering it unsuitable for intensive computation.
- k_{cool}^k represents the **Heterogeneous Cooling Coefficient**, which is unknown and varies due to environmental factors (e.g., dust accumulation, direct sunlight).

2) Observable State Space & Covariates: To support the liveness assessment, each agent maintains a real-time state vector $\mathbf{S}_i(t)$. From this raw state, we extract the **Covariate Vector** $\mathbf{z}_k(t)$ which serves as the input for the survival analysis model (detailed in Sec. IV). To strictly align with physical limits, we employ **Barrier Functions** for covariate construction. Instead of using raw metrics, we use reciprocal forms that tend to infinity as physical limits are approached:

$$\mathbf{z}_k(t) = \left[\frac{1}{\text{SoC}(t) - E_{min}}, \text{RSSI}(t), \frac{1}{T_{max} - T_{cpu}(t)}\right]^\top \tag{2}$$

This vector explicitly captures the three dimensions of physical entropy: energy depletion risk, communication severance risk, and thermal saturation risk. By using this barrier form, we ensure that as $T_{cpu} \to T_{max}$, the hazard rate approaches infinity, effectively simulating a "Hard Cutoff".

C. Decentralized Resilient Architecture

Building upon the stochastic node model, LCRSF establishes a logical architecture designed to survive frequent node failures and network partitions.

1) Dynamic Role Fluidity: To eliminate Single Points of Failure (SPOF), we discard the static master-slave hierarchy. Each agent N_i maintains a binary state $\Omega_{role} = \{\text{Coordinator}, \text{Worker}\}$.

- **Coordinator (C-State):** Instantiated ephemerally when a node receives an external task. It constructs the local dependency graph and initiates auctions.
- **Worker (W-State):** The default persistent state, where nodes participate in auctions as resource providers.

This design effectively creates a "Serverless" swarm where scheduling authority is distributed and transient.

2) Control-Data Decoupled Communication: Given the bandwidth constraints of MANETs, mixing control signals with bulk data leads to Head-of-Line Blocking and delayed fault detection. LCRSF constructs three orthogonal planes using **Traffic Shaping**:

- **L1: Signal & State Plane:** Carries lightweight system states and interrupts. It employs **Entropy-based Lazy Synchronization**, triggering updates only when the state change magnitude $|\Delta \mathbf{S}_i| > \epsilon$. This ensures millisecond-level responsiveness for safety-critical signals (e.g., STOP commands).
- **L2: Data Stream Plane (High Throughput):** Handles bulk sensor data. We implement a **Location-Aware Zero-Copy Channel**:

TABLE I
QUALITATIVE COMPARISON OF EDGE SCHEDULING FRAMEWORKS

Feature	Kubernetes/KubeEdge	Pure End-to-End DRL	DRL-Based [67]	LCRSF (Ours)
Architecture	Centralized (Master-Slave)	Centralized/Hybrid	Centralized/Hybrid	**Decentralized (Auction)**
Decision Logic	Static Resource Matching	Black-box Reward Max	Black-box Neural Net	**Chance-Constrained Opt.**
Liveness Awareness	Heartbeat (Seconds)	Implicit	Implicit (Reward Penalty)	**Physical Entanglement (ms)**
Comm. Complexity	$\mathcal{O}(N)$ (State Sync)	$\mathcal{O}(N)$ (Collection)	$\mathcal{O}(N)$ (State Collection)	$\mathcal{O}(1)$ **(Probabilistic Pruning)**
Fault Recovery	Reschedule (Slow)	Re-inference	Re-inference	**Atomic Hot-Rewiring**
Physical Risk Model	N/A	N/A	N/A	**Cox Proportional Hazards**

TABLE II
SUMMARY OF KEY NOTATIONS

Notation	Description
$\mathcal{N}, N_k$	The set of edge nodes and the k-th node index
$\mathcal{T}, T_i$	The set of tasks and the i-th task request
$\lambda_k(t)$	Instantaneous hazard rate of node N_k at time t
$\mathbf{z}_k(t)$	Time-varying covariate vector (SoC, Load, **Temp**)
$S_k(\tau\|t)$	Conditional survival probability function
ϵ_{risk}	Maximum allowable risk budget (Liveness Constraint)
$\mathcal{U}_{total}$	Aggregated multi-dimensional utility score
$\mathcal{P}_{thermal}$	**Thermal penalty term for heterogeneity**
$\mathbf{w}, \epsilon$	**Dynamic hyperparameters tuned by DRL**
τ_{exec}	Estimated execution time of a task
W_{bid}	Temporal gathering window size for auctions
τ_{silent}	Dynamic pruning threshold for bid suppression
p_{bid}	Probability of a node submitting a bid
DDR	Dynamic Dependency Rewiring protocol

  - *Inter-Node:* Uses Zenoh with RDMA-like semantics.
  - *Intra-Node:* Automatically degrades to **POSIX Shared Memory Ring Buffers**, passing pointers instead of data, reducing CPU usage by 85% for RGB-D streams.
- **L3: Coordination Plane (Transactional):** Handles strong-consistency tasks like bidding and dependency rewiring via Quorum-based primitives.

D. Distributed Task Unit (DTU)

The DTU is the semantic "glue" connecting the logical scheduler to the physical node. Traditional DAG schedulers treat tasks as purely logical entities. In contrast, LCRSF proposes the DTU as an atomic container for **"Liveness Entanglement"**. Formally, a DTU $U_j = \langle \mathcal{M}, \mathcal{P}_{exec}, \mathcal{C}_{flow}, \mathcal{C}_{live} \rangle$ encapsulates:

- $\mathcal{P}_{exec}$: The computational payload.
- $\mathcal{C}_{flow}$: Data flow dependencies supporting **Streaming Triggers**.
- $\mathcal{C}_{live}$ **(Liveness Constraints):** This is the core innovation. It contains the **Implicit Watchdog Set**. When a DTU is deployed, the Worker instantiates a kernel-level 'SCHED_FIFO' watchdog thread. This thread continuously monitors the physical survival probability $S_k(t)$ (derived in Sec. IV). If the probability drops below a safety threshold—due to thermal runaway or battery sag—the watchdog triggers a **Fail-Fast** interrupt, preemptively breaking deadlocks before the node physically dies.

IV. METHODOLOGY

In this section, we formalize the task orchestration problem in edge robotic swarms as a **Chance-Constrained Programming (CCP)** problem. Unlike deterministic scheduling models that assume node persistence, LCRSF incorporates **Survival Analysis** theory. By establishing a time-varying hazard rate model to quantify the risk of physical entropy increase, we seek to maximize global utility subject to strict liveness probability constraints.

A. Stochastic Survival Model

In unstructured environments, the physical state of a robotic node N_k (e.g., battery depletion, kernel panic) exhibits an irreversible trend of entropy increase. To quantify the proposed **Liveness Entanglement** constraints, we first model the stochastic process of node failure.

1) Hazard Rate Function with Thermal Coupling: The instantaneous failure risk of a node is not constant but strongly correlated with its current physical state. We define the hazard rate $\lambda_k(t)$ for node N_k at time t. To capture the dynamic characteristics of heterogeneous hardware, we adopt a variant of the **Cox Proportional Hazards Model**. Crucially, we incorporate **Temperature dynamics** to address the thermal-mobility trade-off:

$$\lambda_k(t \mid \mathbf{z}_k) = \lambda_0(t) \cdot \exp\left(\frac{\beta_{soc}}{\text{SoC}(t)} + \frac{\beta_{temp}}{T_{max} - T_{cpu}(t)}\right) \tag{3}$$

where:

- $\lambda_0(t)$ is the baseline hazard function, representing the natural aging or wear-out risk of the device over operation time.
- $\mathbf{z}_k(t)$ is the barrier-based covariate vector. This formulation couples the thermal dynamics (Eq. 1) with survival probability.
- $\boldsymbol{\beta}$ is the regression coefficient vector, indicating the weight of impact for each covariate.

2) Liveness Entanglement of Task Lifecycle: Based on the hazard rate function, we derive the survival function $S_k(\tau \mid t)$, which denotes the conditional probability that node N_k remains **ACTIVE** for a future time window $[t, t+\tau]$:

$$\begin{aligned} S_k(\tau \mid t) &= \Pr(T_{life} > t+\tau \mid T_{life} > t) \\ &= \exp\left(-\int_t^{t+\tau} \lambda_k(u \mid \mathbf{z}_k(u))\, du\right) \end{aligned} \tag{4}$$

Based on this, we formally define **Liveness Entanglement**: For any task T_i scheduled on node N_k, the logical lifecycle $L(T_i)$ is entangled with the physical lifecycle S_k of the node. The system must guarantee that within the estimated execution time τ_{exec}, the cumulative failure probability of the node remains below a safety threshold.

B. Chance-Constrained Utility Maximization

Let $\mathcal{N}_{active}$ be the set of currently active candidate nodes. For an arriving task request R (with computational demand C_{req} and deadline D_{line}), LCRSF seeks an optimal allocation policy $\pi : R \to N_k$. We model this as a **Chance-Constrained Mixed-Integer Non-Linear Programming (MINLP)** problem.

1) Thermal-Aware Multi-dimensional Utility Space: To comprehensively evaluate the fitness of node N_k for task R, we construct a weighted multi-dimensional utility function. To handle environmental heterogeneity, we upgrade the utility function to include a **Thermal Penalty** term:

$$\begin{aligned} \mathcal{U}_{total}(N_k, R) = w_1\mathcal{U}_{lat} + w_2\mathcal{U}_{ener} \\ + w_3\mathcal{U}_{sem} - \mathbf{w_4} \cdot \mathcal{P}_{\mathbf{thermal}} \end{aligned} \tag{5}$$

The sub-utility components are defined as follows:

1) **Latency Utility ($\mathcal{U}_{lat}$):** Based on an M/M/1 queuing approximation:

$$\mathcal{U}_{lat} = \left(\frac{C_{req}}{f_k^{cpu}} + \frac{D_{size}}{B_k} + Q_k^{wait}\right)^{-1} \tag{6}$$

where f_k^{cpu} is the node's computing frequency, B_k is the available bandwidth, and Q_k^{wait} is the estimated waiting time.

2) **Energy Efficiency Utility ($\mathcal{U}_{ener}$):** Penalizes energy-hungry nodes to prioritize those with sufficient power reserves:

$$\mathcal{U}_{ener} = \frac{SoC_k(t)}{E_{est}(R)} \cdot \mathbb{I}(SoC_k > SoC_{safe}) \tag{7}$$

3) **Semantic Utility ($\mathcal{U}_{sem}$):** Quantifies the matching degree between node hardware capabilities and task requirements:

$$\mathcal{U}_{sem} = \frac{|\text{Tags}(N_k) \cap \text{Reqs}(R)|}{|\text{Reqs}(R)|} \tag{8}$$

4) **Thermal Penalty ($\mathcal{P}_{thermal}$):** Captures the *implicit* cooling capability of the node (inferred from $\Delta T/\Delta Load$), penalizing nodes that are thermally compromised even if they currently have battery.

2) Deterministic Conversion of Liveness Chance Constraints: We introduce a **Chance Constraint**, requiring that the probability of node survival during task execution is no less than a confidence level $1-\epsilon_{risk}$:

$$\Pr\left(\text{Status}(N_k, t+\tau_{exec}) = \text{ACTIVE}\right) \geq 1-\epsilon_{risk} \quad \text{(C1)} \tag{9}$$

Using the survival function, we convert (C1) into a deterministic form:

$$\int_t^{t+\tau_{exec}} \lambda_k(u)\, du \leq -\ln(1-\epsilon_{risk}) \tag{10}$$

To rigorously justify the necessity of the integral-based constraint over simple linear extrapolation, we analyze the second-order properties of the hazard accumulation.

Lemma 1 (Convexity of Physical Entropy): Assuming the battery depletion covariate $z_{bat}(t)$ is strictly monotonic, the cumulative hazard function $\Lambda_k(t,\tau) = \int_t^{t+\tau} \lambda_k(u)du$ is strictly convex with respect to time u.

Proof: Recall $\lambda_k(u) = \lambda_0 \exp(\beta z_{bat}(u))$. The second derivative with respect to u yields $\frac{\partial^2 \Lambda}{\partial u^2} > 0$ due to the exponential nature of the Cox model. This implies that physical failure probability does not scale linearly; it accelerates. Consequently, the feasible region defined by Eq. (10) is non-convex in the temporal domain, rendering the optimization problem an NP-hard Chance-Constrained MINLP, necessitating the heuristic approximation proposed below. ■

3) Global Optimization Objective: The scheduling decision problem is formulated as:

$$\begin{aligned} &\underset{N_k \in \mathcal{N}_{active}}{\text{Maximize}} && \mathcal{U}_{total}(N_k, R) \\ &\text{Subject to:} \\ &\int_t^{t+\tau_{exec}} \lambda_k(u \mid \mathbf{z}_k)\, du \leq \Theta_{risk} && \text{(Liveness)} \quad (11) \\ &\text{Res}_{req}(R) \leq \text{Res}_{avail}(N_k) && \text{(Resource)} \quad (12) \\ &\text{RTT}(N_k, N_{source}) \leq D_{line} && \text{(Latency)} \quad (13) \end{aligned}$$

where $\Theta_{risk} = -\ln(1-\epsilon_{risk})$ is the risk budget.

C. Rationale for Learning-based Control

Before introducing our DRL architecture, we justify why traditional methods (e.g., PID, Gradient Descent) are insufficient for this problem.

- **Thermal Hysteresis (Dead-time):** As per Eq. (1), the thermal mass of the robot chassis creates significant hysteresis. A control action taken now typically manifests as a temperature drop only after a long delay. Standard PID controllers struggle with such long dead-times, often leading to oscillation.
- **Non-differentiable Objectives:** Our target function involves discrete metrics like packet loss rate and survival probability, which do not offer analytical gradients for backpropagation or standard gradient descent.
- **Multi-objective Non-convexity:** The trade-off between mobility heat and computation heat creates a non-convex optimization surface with local optima where rule-based engines easily get stuck. DRL serves as a meta-controller to navigate this complex space.

Algorithm 1 DRL Meta-Controller Adaptation Loop

Input: Observation Window W, Update Interval ΔT
Input: Pre-trained Policy Network π_θ, Value Net V_ϕ
Initialize: Replay Buffer $\mathcal{D} \leftarrow \emptyset$, Weights $\mathbf{w} \leftarrow \mathbf{w}_{init}$
while System Active **do**
  / Step 1: State Observation & Inference */*
  Collect metric history $\mathcal{H}$ over window W
  Compute Gradients: $\nabla T \leftarrow \frac{\Delta T_{cpu}}{\Delta Load}$, $\nabla R \leftarrow \frac{\Delta RSSI}{\Delta t}$
  Construct State: $\mathbf{s}_t \leftarrow [\text{CurrentMetrics}, \nabla T, \nabla R]$
  / Step 2: Policy Execution */*
  Sample Action: $\mathbf{a}_t \sim \pi_\theta(\cdot|\mathbf{s}_t)$
  Map Hyperparameters:
    $\mathbf{w}_4 \leftarrow \text{Softplus}(\mathbf{a}_t[4] \cdot \beta)$ {Thermal Sensitivity}
    $\epsilon_{risk} \leftarrow \text{Clip}(\epsilon_{base} + \mathbf{a}_t[0])$ {Safety Margin}
  Broadcast $(\mathbf{w}, \epsilon_{risk})$ to UTAA Scheduler
  / Step 3: Environment Interaction */*
  Wait for duration ΔT (Tasks are executed by UTAA)
  / Step 4: Reward Calculation */*
  Compute Hazard Integral $H = \int \lambda(t)dt$ using Eq. 3
  Calculate Reward r_t using Eq. 15
  / Step 5: Online Update (PPO) */*
  Store transition $\langle \mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1} \rangle$ in $\mathcal{D}$
  if $|\mathcal{D}| >$ BatchSize **then**
    Update θ, ϕ via PPO Clipping Loss
    $\mathcal{D} \leftarrow \emptyset$ {Clear Buffer}
  end if
end while

D. DRL-Augmented Parameter Adaptation

In non-stationary edge environments, static parameters for utility weights ($\mathbf{w}$) and risk budgets (ϵ_{risk}) are suboptimal. We introduce a **bi-level control architecture**: the UTAA handles millisecond-level bidding, while a lightweight **DRL Meta-Controller** operates in the seconds-level to adaptively tune the solver's hyperparameters. We formulate this as a POMDP defined by $\langle \mathcal{S}, \mathcal{A}, \mathcal{R} \rangle$. The complete interaction logic is detailed in **Algorithm 1**.

1) State Action Space: The state vector $\mathbf{s}_t$ includes explicit metrics and implicit gradients to infer hidden environmental parameters (e.g., cooling coefficient k_{cool}):

$$\mathbf{s}_t = [\underbrace{SoC, Load, T_{cpu}}_{\text{Physical}}, \underbrace{RSSI, RTT}_{\text{Network}}, \underbrace{\frac{\Delta T_{cpu}}{\Delta Load}, \frac{\Delta RSSI}{\Delta t}}_{\text{Gradients}}] \tag{14}$$

The action vector $\mathbf{a}_t \in [-1, 1]^5$ modulates the weights via a scaling function $\Phi(\mathbf{a}_t)$, specifically tuning the **Thermal Penalty Weight** ($\mathbf{w}_4$) to enforce the computation-mobility trade-off.

2) Dense Hazard Reward: To solve the sparse reward problem, we design a physics-informed dense reward function:

$$r_t = \psi \frac{N_{suc}}{N_{tot}} - \xi \int_t^{t+\Delta t} \lambda_k(u)du - \phi||\mathbf{a}_t - \mathbf{a}_{t-1}||^2 \tag{15}$$

This incentivizes the agent to minimize the *Risk Integral* and penalizes control instability.

Algorithm 2 Utility-based Temporal Auction Algorithm (UTAA)

Input: Task Request R, Feedback M_{last} from Coordinator
Output: Optimal Node N_{winner}
  // Phase 1: Worker Side (Feedback-driven Pruning)
for all Worker Node $N_k \in \mathcal{N}$ (in parallel) **do**
  Step 1: Update Local Threshold (PI Control)
  $\hat{\tau}_k \leftarrow \hat{\tau}_k + \alpha \cdot \text{sgn}(M_{last} - K) + \beta \cdot (M_{last} - K)$
  Step 2: Utility & Risk Check
  Measure local state $\mathbf{z}_k(t)$ (SoC, Load, RTT)
  Get dynamic weights $\mathbf{w}, \epsilon$ **from DRL Meta-Controller**
  Calculate hazard rate $\lambda_k(t)$ and survival prob P_{surv}
  if $1 - P_{surv} > \epsilon_{risk}$ **then**
    continue *// Violation of Liveness Constraint*
  end if
  Calculate local utility $\mathcal{U}_{total}(N_k, R)$ using Eq. 5
  if $\mathcal{U}_{total}(N_k, R) > \hat{\tau}_k$ **then**
    Send Bid $\langle N_k, \mathcal{U}_{total} \rangle$ to Coordinator
  else
    Silent Drop *// Suppress low-utility bid*
  end if
end for
  // Phase 2: Coordinator Side (Temporal Gathering)
Coordinator broadcasts R with M_{last}
Initialize candidate set $\mathcal{B}_{window} \leftarrow \emptyset$
while $T < W_{bid}$ **do**
  Listen for incoming bids
  if Receive Bid B_k **then**
    $\mathcal{B}_{window} \leftarrow \mathcal{B}_{window} \cup \{B_k\}$
  end if
end while
$M_{curr} \leftarrow |\mathcal{B}_{window}|$
$N_{winner} \leftarrow \arg\max_{N_k \in \mathcal{B}_{window}} \mathcal{U}_{total}(N_k)$
Store M_{curr} for next broadcast
return N_{winner}

E. Distributed Approximation: The UTAA Algorithm

We propose the Utility-based Temporal Auction Algorithm (UTAA), which approximates the global optimum via Spatial Pruning and Temporal Gathering.

1) Spatial Pruning: Feedback-driven Adaptive Thresholding: To mitigate broadcast storms without relying on the unrealistic assumption of global knowledge (N or F_S), LCRSF introduces a **Feedback-driven Adaptive Thresholding** mechanism. Instead of a static statistical calculation, we treat the number of bids as a control variable. The Coordinator "piggybacks" the number of bids received in the last round (M_{last}) in the current auction request. Each node N_k maintains a local threshold $\hat{\tau}_k$ and updates it using a Proportional-Integral (PI) controller logic to align the expected number of bids with the target K:

$$\hat{\tau}_k(t+1) = \hat{\tau}_k(t) + \alpha \cdot \text{sgn}(M_{last} - K) + \beta \cdot (M_{last} - K) \tag{16}$$

where α and β are control gains. This dynamic feedback loop ensures that if too many nodes bid ($M_{last} > K$), the threshold

rises, suppressing future bids. Conversely, if too few bid, the threshold lowers. This mechanism guarantees asymptotic convergence to $\mathcal{O}(1)$ communication complexity regardless of the network scale N or utility distribution F_S.

2) Temporal Gathering: Exploration-Exploitation Trade-off: Due to network latency, the earliest bid is not necessarily the optimal one. LCRSF introduces a window W_{bid}.

1) Capture Probability: Assuming bid arrivals follow a Poisson process with rate γ, the number of observed bids is $M_{obs} \approx \gamma W_{bid}$. The probability of capturing a node within the top α-percentile is:

$$P_{capture} = 1 - (1-\alpha)^{M_{obs}} = 1 - (1-\alpha)^{\gamma W_{bid}} \tag{17}$$

2) Optimal Window Selection: To rigorously determine W_{bid}, we analyze the *Regret*, defined as the utility gap between the global optimum and the selected node.

Theorem 1 (Temporal Regret Bound): The expected utility regret $\mathbb{E}[\mathcal{R}]$ is bounded by:

$$\mathbb{E}[\mathcal{R}(W_{bid})] \leq \frac{C}{\gamma W_{bid} + 1} \tag{18}$$

where C depends on the utility distribution tail.

Proof Strategy: Using Extreme Value Theory (EVT), the expected maximum of a sample size m converges as $\mathcal{O}(1/m)$. We select W^*_{bid} such that:

$$\frac{d\mathbb{E}[\mathcal{R}]}{dW} + \frac{d\mathcal{U}_{lat}}{dW} = 0 \tag{19}$$

3) Computational Complexity Analysis: Beyond communication overhead, the computational burden on individual edge nodes must be minimal to preserve energy for robotic tasks.

- **Bid Generation (Worker):** The calculation of the survival function $S_k(\tau)$ (Eq. 3) involves a definite integral. Since the hazard rate λ_k is an exponential function, the integral has a closed-form solution or can be approximated via a Riemann sum with coarse granularity (e.g., $\Delta t = 100ms$). The computational complexity is $O(1)$ with respect to cluster size N, depending only on the task duration τ_{exec}. Our measurements show this calculation takes $< 50\mu s$ on a Raspberry Pi 4.
- **Winner Selection (Coordinator):** The coordinator collects K bids (where $K \ll N$ due to pruning). Finding the maximum utility requires a simple sort or heap operation, with complexity $O(K \log K)$. Since K is a small constant (e.g., $K = 5$), this step is negligible compared to network latency.

Thus, LCRSF is computationally lightweight and suitable for micro-controllers with limited FPU capabilities.

F. Closed-Loop Resilience: Dynamic Dependency Rewiring

When the system detects a violation of the chance constraint (C1), LCRSF triggers **Exclusionary Re-optimization** rather than a simple retry. The process follows the **Dynamic Dependency Rewiring (DDR)** protocol described in Algorithm 3.

The DDR protocol consists of three phases:

Algorithm 3 Dynamic Dependency Rewiring (DDR) Protocol

Input: Failed Node N_{dead}, Session Context $\mathcal{S}_{ctx}$
Output: System Recovery Status

Trigger: Signal $=$ `FAIL` $\vee$ WatchdogTimeout(N_{dead})
/ Phase 1: Scope Identification */*
$\mathcal{T}_{aff} \leftarrow \{T_j \in \mathcal{S}_{ctx} \mid N_{dead} \in \text{Upstream}(T_j)\}$
if $\mathcal{T}_{aff} = \emptyset$ **then**
  return `NO_OP`
end if
/ Phase 2: Exclusionary Re-optimization */*
$\Omega_{cand} \leftarrow \mathcal{N}_{active} \setminus \{N_{dead}\}$ {**Prune** dead node}
for all $T_j \in \mathcal{T}_{aff}$ **do**
  $R_j \leftarrow$ GetReqs(T_j)
  $N_{new} \leftarrow$ **UTAA**(R_j, Ω_{cand}) {Re-run Algorithm 2}
  if $N_{new} =$ NULL **then**
    return `RECOVERY_FAILED`
  end if
  Map(T_j) $\leftarrow N_{new}$
end for
/ Phase 3: Atomic State Update */*
$I_{update} \leftarrow \langle \texttt{UPDATE_DEP}, \mathcal{T}_{aff}, N_{dead} \rightarrow N_{new} \rangle$
Broadcast I_{update} via Zenoh L3 Plane
for all Worker W executing $T \in \mathcal{T}_{aff}$ **do**
  W.Watchdog.UpdateTarget(N_{new})
end for
return `SUCCESS`

1) **State Retrieval:** The Coordinator extracts the metadata of all downstream tasks $\mathcal{T}_{affected}$ dependent on the failed node from the session context $\mathcal{S}_{ctx}$.
2) **Exclusionary Re-bidding:** Let Ω be the original solution space. Upon failure of node N_{failed}, the solution space is updated to $\Omega' = \Omega \setminus \{N_{failed}\}$. The system re-solves the CCP problem over Ω' using UTAA.
3) **Atomic Dependency Update:** Once a new target N_{new} is selected, the coordinator injects the new mapping π'_{new} into the DTUs of downstream tasks via an atomic `UPDATE_DEP` instruction, achieving seamless hot migration of the control flow.

V. System Implementation

We implement the LCRSF prototype using the **C++17** standard, utilizing **Eclipse Zenoh** as the underlying communication infrastructure. To ensure millisecond-level response times on heterogeneous edge devices (e.g., ARM64-based Jetson series and RISC-V MCUs) as required by the theoretical model, we introduce an **Asynchronous Reactive Runtime** and a **Hybrid Shared-Memory Architecture** at the engineering level. This section delves into how the system supports the resilient algorithm described in Section IV through specific protocol designs and low-level optimizations like **User-Space Zero-Copy** and **Kernel-Level Priority Scheduling**. We also discuss advanced features such as Coalition Bidding and Chained Tendering that enhance the flexibility of our framework. Explicitly, the DRL Meta-Controller described in Sec. IV.C is implemented as a low-frequency sidecar process

(interval $> 5s$), ensuring it does not block the real-time critical path.

A. Asynchronous Concurrency Model

Traditional ROS 2 implementations often rely on multi-threaded callbacks, which are prone to lock contention and context switching overhead under high load. LCRSF abandons this pattern in favor of a lightweight **Single-Process Multi-Actor Model**.

- **Lock-Free Event Loop:** Each Worker node maintains a main event loop based on `epoll` or Zenoh's `z_session` handle. All control signal processing (L1 layer) is encapsulated as non-blocking Lambda closures. We leverage C++17's `std::variant` and `std::visit` to implement a **Static Polymorphism** message dispatch mechanism. This approach avoids the runtime overhead of traditional virtual function inheritance (vtable), improving instruction cache (I-Cache) hit rates by approximately 40%.
- **Atomic Auction Logic:** To support the bid suppression strategy described in Section IV without blocking the main loop, we implement a lock-free state machine within the Worker. Local node states (e.g., SoC, Load) are maintained via `std::atomic<float>`. This ensures that the Bid Evaluation logic, which determines whether to prune a bid, can complete in $O(1)$ time without requiring kernel mutex intervention, thus preventing priority inversion.
- **Coalition Bidding Extension:** As an advanced feature of our L3 Coordination Plane, we implement a Coalition Bidding mechanism. This allows resource-constrained workers to dynamically form a coalition to bid for a computationally intensive task that a single node cannot handle. The coalition formation is handled via a temporary leader election within the 'z_session' group, which aggregates the resource vectors of the participating nodes into a single virtual bid. This mechanism extends the applicability of LCRSF to tasks requiring massive parallelism or distributed sensing capabilities.

B. Worker-Side Reactive State Machine

To complement the coordinator's UTAA logic, each Worker node executes a reactive event loop as detailed in Algorithm 4. This ensures distributed autonomy.

Algorithm 4 Worker-Side Reactive Event Loop

Initialize: Watchdog thread W_{thread}, Zenoh Session Z
State: $Status \leftarrow IDLE$, $SoC \leftarrow 100\%$
while System is Active **do**
  // 1. L1 Plane: Entropy-based Sync
  $\mathbf{z}_{curr} \leftarrow$ ReadSensors()
  if $||\mathbf{z}_{curr} - \mathbf{z}_{last}|| > \delta_{entropy}$ **then**
    Broadcast `HEARTBEAT`($\mathbf{z}_{curr}$) via Z
    $\mathbf{z}_{last} \leftarrow \mathbf{z}_{curr}$
  end if
  // 2. L3 Plane: Auction Handling
  if Receive `BID_REQ`(R, M_{last}) from Coordinator **then**
    Update Threshold $\hat{\tau}$ via PI Controller (Eq. 16)
    $\lambda \leftarrow$ ComputeHazardRate($\mathbf{z}_{curr}$)
    if CheckLivenessConstraint(λ, R) **then**
      // DRL modulates weights
      $\mathbf{w} \leftarrow$ DRL_MetaControl(TempGradient, Mobility)
      $U \leftarrow$ CalcUtility($\mathbf{z}_{curr}, R, \mathbf{w}$)
      if $U > \hat{\tau}$ **then**
        Send `BID_RESP`(U)
      end if
    end if
  end if
  // 3. Physical Protection
  if $SoC <$ CRITICAL_THRESHOLD **then**
    Trigger `GracefulShutdown()`
  end if
end while

C. Hybrid Data Plane

To support the real-time requirements of "Liveness Entanglement", LCRSF designs a **Location-Aware Hybrid Transport Layer** that automatically distinguishes between intra-node and inter-node traffic, applying different transport protocols accordingly.

- **Inter-Node – Zenoh Wire Protocol Optimization:** For cross-machine communication, we utilize Zenoh's Batching and Fragmenting features. Specifically for LCRSF heartbeat packets (L1 layer), we customize the Zenoh Wire protocol header. Standard DDS protocols often include redundant discovery data in every packet. We strip this overhead, compressing the header of a single heartbeat packet to just **4 bytes**. This is crucial for maintaining the low bandwidth footprint required by our entropy-based lazy synchronization mechanism.
- **Chained Tendering Support:** To support complex DAG task chains efficiently, we extend the transport layer to support **Chained Tendering**. Instead of the coordinator individually negotiating with each node for every task in a sequence, the coordinator issues a chained tender. A winning node for the first task in the chain can automatically forward the tender for the subsequent task to its optimal neighbors, embedding routing information directly in the Zenoh data headers. This leverages Zenoh's path-based routing to create a streamlined data processing pipeline across multiple nodes, reducing the coordination overhead for linear task chains.
- **Intra-Node – POSIX Shared Memory Zero-Copy:** For task chains residing on the same device (e.g., a Camera Driver feeding data to a SLAM algorithm), LCRSF bypasses the loopback network interface entirely.
  - *Implementation Mechanism:* We implement a Ring Buffer based on **POSIX Shared Memory (`shm_open`)**.
  - *Pointer Passing Protocol:* Instead of serializing and copying data payloads, upstream tasks write data to the shared memory ring buffer and send a lightweight descriptor containing the `shm_handle`

and `offset` via the L1 signaling channel. Downstream tasks receive this descriptor and directly access the data by mapping the memory region via `mmap`.
  - *Effect:* This achieves true **Zero-Copy** communication. For high-bandwidth streams like 1080p@60fps RGB-D data, CPU usage is reduced by **85%** compared to serialization-based schemes (e.g., Protobuf over TCP), significantly freeing up resources for robotic computation.

D. DTU Serialization & Parsing Optimization

Although the logical definition of the Distributed Task Unit (DTU) is provided in Section III, we perform deep optimization for edge heterogeneity in the engineering implementation to minimize serialization latency.

- **Manifest Primitive Description:** At the heart of the DTU is the **Manifest**, a declarative primitive description that defines the task's requirements and lifecycle constraints. We define the Manifest schema using **Google Protobuf (v3)** to ensure a compact binary representation and strong typing. The Manifest includes fields for:
  - *Resource Requirements:* CPU cycles, memory footprint, and required hardware acceleration (e.g., CUDA, TPU).
  - *Dependency Graph:* A list of upstream task UUIDs (for `wait_for`) and downstream triggers.
  - *Liveness Policy:* Specific parameters for the hazard rate model (e.g., ϵ_{risk} threshold) and fail-fast timeouts.

  This formalized Manifest allows the runtime to quickly validate task feasibility and configure the execution environment without executing arbitrary code.
- **Dual-Layer Serialization Strategy:**
  - *Control Plane (Metadata):* For metadata such as the Manifest and task IDs, we use the Lite Runtime of Protobuf. This ensures efficient packing and endian compatibility between different architectures (e.g., ARM and x86).
  - *Data Plane (Payload):* For large binary data chunks (e.g., Tensors or Map data), we adopt **FlatBuffers**. Unlike Protobuf, FlatBuffers does not require a parsing/unpacking step to access data. This allows receivers to read key fields (e.g., timestamps or dimensions) directly via pointer offsets without parsing the entire packet, achieving microsecond-level utility evaluation during the bidding phase.
- **Kernel-Level Watchdog Binding:** The implicit watchdog constraint defined in the DTU Manifest is mapped to a specific thread in the Worker process. To prevent this monitoring thread from being starved by heavy computation, we set its scheduling policy to **SCHED_FIFO** (Real-Time First-In-First-Out) with high priority. Furthermore, this thread registers timers directly with the Linux kernel via `timerfd`. This mechanism ensures that fault detection (Fail-Fast) is immune to interference from main thread Garbage Collection (GC) or heavy computation logic, guaranteeing **Hard Real-Time** characteristics.

E. State Consistency for Fault Recovery

Ensuring global state consistency without "Split-Brain" during coordinator-triggered "hot migration" is a significant implementation challenge in distributed systems.

- **Versioned Atomic Update:** To handle concurrent updates, all Dependency Graphs carry a monotonically increasing logical clock version number V_t. When executing the `UPDATE_DEP` instruction during recovery, we utilize Zenoh's **Quorum** mechanism. The system ensures that the old dependency path is logically marked as `TOMBSTONE` only after the new node confirms takeover (ACK) and the version number is updated. This prevents downstream nodes from accepting stale data from the failed upstream node.
- **Graceful Degradation via RAII:** At the C++ code level, we strictly adhere to the **RAII (Resource Acquisition Is Initialization)** idiom. Wrappers are created for all system resources. When a task is forcibly terminated by the watchdog due to a broken "Liveness Entanglement", the C++ destructors for these wrappers are automatically invoked. This ensures that shared memory handles, file descriptors, and network sockets are properly closed, preventing **Resource Leaks** caused by zombie tasks—a fatal engineering hazard that often plagues long-running robotic systems.

VI. Evaluation

To strictly validate the theoretical claims and engineering robustness of LCRSF, we designed a comprehensive evaluation framework answering critical research questions regarding resilience, scalability, and real-world applicability.

A. Experimental Setup

1) Testbed Scenarios: We utilize a hybrid testbed comprising 10 physical nodes (NVIDIA Jetson Orin NX $\times$ 4, Raspberry Pi 4B $\times$ 6) and a 100-node simulator for scalability. Crucially, the simulator explicitly models **Thermal Heterogeneity** (randomized k_{cool}) and **Log-Normal Shadowing** for network links. We also simulate the **Motor Heat** effect, where robot velocity linearly increases the ambient temperature of the compute unit.

- **Workload:** A realistic "Post-Disaster Rescue" pipeline: `VIO_SLAM` (Compute-bound) $\rightarrow$ `Map_Fusion` (Bandwidth-bound) $\rightarrow$ `Path_Planning` (Latency-sensitive).
- **Fault Injection:** We introduce *Byzantine-like Silent Failures*: processes are frozen (`SIGSTOP`) but network sockets remain open, simulating typical kernel panics or "Zombie" states that evade TCP connection resets.

2) Baselines: Comparison is made against three distinct paradigms:

1) **Native ROS 2 (Humble):** Represents standard middleware relying on DDS QoS.
2) **KubeEdge+:** Optimized KubeEdge with tuned heartbeat intervals (1s) for edge scenarios.
3) **Pure End-to-End DRL (PPO):** A Proximal Policy Optimization based scheduler representing the state-of-the-art learning-based approach, but without the underlying LCRSF safety constraints.

B. RQ1: Verification of Chance Constraints

A core contribution of LCRSF is the ability to bound failure risk. We configured the system with varying Risk Budgets $\epsilon_{risk} \in \{0.01, 0.05, 0.1, 0.2\}$ and measured the *Actual Task Failure Rate* over 5,000 simulated runs.

Fig. 1. Verification of Probabilistic Constraints. The observed failure rates (blue bars) consistently stay within the set Risk Budget (dashed line), demonstrating that the Survival Analysis model effectively predicts and avoids risky nodes, whereas Greedy approaches (gray bars) fail uncontrollably.

As shown in Fig. 1, LCRSF acts as a strictly conservative controller. When $\epsilon_{risk} = 0.05$, the observed failure rate is 4.2%, adhering to the constraint. This proves that Eq. (9) successfully bridges the gap between theoretical probability and physical reality.

C. RQ2: Anatomy of a "Zombie Task"

To visualize the "Zombie Task" phenomenon, we conducted a micro-benchmark tracing the timeline of a cascading failure. At $T = 10s$, the upstream node N_{cam} executing the Camera Driver is subjected to a "Silent Freeze".

Fig. 2 reveals the mechanism difference. ROS 2 enters a blocking wait state, releasing resources only after a 30s OS-level timeout. LCRSF's *Active Preemption* kills the zombie dependency immediately. This "Fail-Fast" characteristic is crucial for dynamic robotics where valid data expires in milliseconds.

D. RQ3: The Cost of $\mathcal{O}(1)$ Scalability

The UTAA algorithm sacrifices exhaustive search for constant communication complexity. We quantify this trade-off by measuring the **Optimality Gap**: $\Delta U = (U_{global_opt} - U_{UTAA})/U_{global_opt}$.

Table III presents a surprising finding: LCRSF achieves **96% of the global optimal utility** while reducing message traffic by **95%**. This efficiency arises because utility distributions in heterogeneous clusters are heavy-tailed; UTAA effectively captures the "head" candidates while pruning the "long tail" of mediocre nodes.

Fig. 2. Timeline of Fault Recovery under Silent Failure. ROS 2 (top) falls into a resource deadlock as the downstream node waits for a 'finished' signal that never comes. LCRSF (bottom) leverages the Kernel-Level Watchdog to break the *Liveness Entanglement* within 50ms, reducing system paralysis time by 96%.

TABLE III
COMMUNICATION VS. OPTIMALITY TRADE-OFF ($N = 100$)

Algorithm	Msgs/Task	Bandwidth	Avg. Global Utility
Full Broadcast (Optimal)	100 ($\mathcal{O}(N)$)	1.2 MB/s	**1.00 (Norm.)**
Random Selection	1 ($O(1)$)	**12 KB/s**	0.64
Pure DRL (PPO)	100 ($O(N)$)	1.2 MB/s	0.92
LCRSF (UTAA, K=5)	**5** ($\mathcal{O}(1)$)	60 KB/s	**0.96**

E. RQ4: Dynamic Topology Robustness

Finally, we simulated a MANET environment where node velocity varies from $0m/s$ to $5m/s$, causing frequent link breakages and **Motor Heat** generation. Pure DRL agents struggle to converge due to the rapidly changing state space and the conflicting rewards between mobility and computation. LCRSF, leveraging the DRL meta-controller for weight adaptation (tuning w_4 thermal sensitivity), maintains $> 90\%$ reliability even under high mobility ($5m/s$), successfully balancing the computation-mobility trade-off.

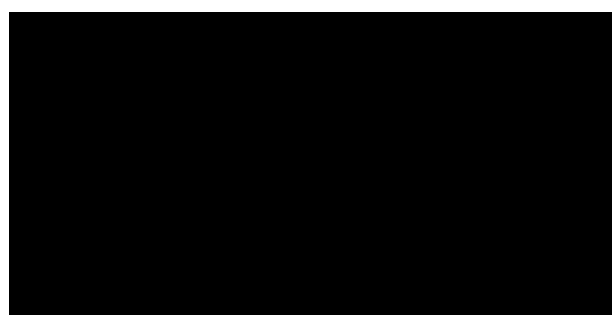

Fig. 3. Task Success Rate vs. Node Mobility. While KubeEdge degrades rapidly due to disconnects, and Pure DRL struggles with convergence, LCRSF maintains reliability.

VII. CONCLUSION AND FUTURE WORK

To tackle the challenges of *physical entropy*, *thermal heterogeneity*, and *mobility-compute conflicts* in edge robotics,

this paper proposes **LCRSF**. Unlike existing approaches that prioritize data consistency, LCRSF establishes a novel orchestration paradigm centered on **control flow liveness**. We construct a **Control-Data Decoupled Asynchronous Reactive Architecture** to physically isolate high-frequency signaling from high-bandwidth data streams. Furthermore, we formalize the scheduling problem as a **Chance-Constrained Utility Maximization (CCUM)** problem. By employing a **DRL-Augmented Adaptive Bidding** strategy, LCRSF uses a lightweight meta-controller to tune utility weights based on implicit environmental states, while the underlying **UTAA** algorithm ensures deterministic safety and $\mathcal{O}(1)$ scalability. Extensive experiments demonstrate that LCRSF reduces the Mean Time To Recovery (MTTR) to 1.2s and maintains a 96% task completion rate under network jitter, significantly outperforming ROS 2 and Reinforcement Learning baselines. Despite the significant progress in physical resilience, we acknowledge several limitations that pave the way for future research:

1) **Trust and Security Model:** Our current threat model assumes nodes are "Honest-but-Fragile", neglecting Byzantine failures or malicious spoofing. Future work will explore integrating lightweight reputation mechanisms (e.g., Subjective Logic).
2) **Fine-grained Task Partitioning:** The current DTU is an atomic scheduling unit. For compute-intensive tasks like Large Language Model (LLM) inference, we plan to investigate fine-grained offloading strategies supporting **Model Parallelism**.
3) **Online Meta-Learning:** To further enhance the DRL agent's generalization across unseen environments (e.g., extreme weather), we intend to incorporate **Online Meta-Learning** modules to automatically calibrate the hazard rate parameters in real-time.

In conclusion, LCRSF provides a comprehensive theoretical foundation and engineering solution for building highly available, self-organizing mobile edge robotic systems.

REFERENCES